

Generating the polynomials of the search

Section 5 of Fortin, Gouicem and Graillat in Rocq: a guide to `code/APaul/rocq/Shift.v` and `code/APaul/rocq/ShiftExp.v`

Names set in `monospace` are the Rocq identifiers of the two files.

1 The function f

Everything in section 5 is about a function

$$f : \mathbb{N} \rightarrow V$$

where V is any abelian group (a `zmodType` in Rocq). So f is a sequence: it has a value at $0, 1, 2, \dots$ and nowhere else. Between n and $n + 1$ there is nothing, so asking whether f is continuous or differentiable makes no sense. The only things we can do with f are read its values and add or subtract them, and that is all section 5 needs. This is the setting of `Shift.v`.

An example: the exponential. Here is how the search gets such a function. It starts with a function on the reals and ends with a sequence of integers.

1. $\exp : \mathbb{R} \rightarrow \mathbb{R}$, the function whose hard cases we look for.
2. $P_{\mathbb{R}} : \mathbb{R} \rightarrow \mathbb{R}$, a polynomial of degree 7 that approximates $\exp$ near the search interval (`P_R`, in `Cheb.v`).
3. $P : \mathbb{N} \rightarrow \mathbb{Z}$, the values of $P_{\mathbb{R}}$ at the arguments of the search. The n -th argument is $x_n = x_0 + n/2^{54}$. Scaled by 2^{598} , $P_{\mathbb{R}}(x_n)$ is an integer, and that integer is $P(n)$ (`Pdir`, with `Pdir_chebE` proving the equality).

The first two are differentiable, but section 5 never uses that. The third one is our f , with $V = \mathbb{Z}$. This is the setting of `ShiftExp.v`.

2 The problem

The hard-to-round search does not work on the exponential. It works on a polynomial that approximates it. The search walks the arguments in intervals, and on each interval it needs a polynomial of its own: if P approximates the function on the whole range and an interval starts at argument kN , the search needs $P_k(m) = P(kN + m)$.

In `htr.c` an interval holds $N = 2^{20}$ arguments, and the search interval $[0.25, 0.25001)$ holds 180 143 985 095 of them. That is 171 799 interval polynomials. Computing an approximation afresh for each one would cost more than the search.

Section 5 of the paper builds all of them from a single polynomial, using **additions only**. Two things make this possible: a polynomial of degree d is determined by $d + 1$ numbers, and those numbers can be moved from one interval to the next by adding them to each other.

Nothing in the section rounds. The coefficients are fixed-point integers and every step is exact. So the whole of `Shift.v` is stated over an arbitrary abelian group, with no floating point anywhere in it. The approximation error enters once, at the very end, and is the subject of section 9.

3 The difference operator

The paper's Δ is

```
Definition dif f : nat -> V := fun n => f n.+1 - f n.  
Definition difn i f : nat -> V := iter i dif f.
```

It is a finite difference, not a derivative: there are no limits, only a subtraction of two neighbouring values. For example, with $f(n) = n^2$ we get $\Delta f(n) = (n+1)^2 - n^2 = 2n+1$, then $\Delta^2 f(n) = 2$ and $\Delta^3 f(n) = 0$. Each application of Δ lowers the degree by one, as a derivative would, but it is computed with one subtraction.

A polynomial is a function from arguments to values, sampled at the integers. We never need polynomials as a datatype, and we never need their coefficients: the only thing we need is the degree, and the degree is this.

Definition `degle d f := forall n, difn d.+1 f n = 0.`

A function has degree at most d when its $(d+1)$ -st difference vanishes everywhere. For a polynomial in the ordinary sense this is the usual degree. Every statement below uses this one and no other.

Two remarks on the Rocq. First, there is no functional extensionality in either file. `difn i f n` reads f at finitely many points, so a pointwise equality between two functions is enough to rewrite underneath it; that is `difn_ext`, and it is proved by induction like everything else. Second, `dif` is an operator on functions, not on a syntax of polynomials, so the file needs no polynomial library at all.

4 Newton's formula

Everything in section 5 comes from one identity.

Lemma `dif_shiftn f n k :`
`f (n + k)%N = \sum_(0 <= i < k.+1) (difn i f n) *+ 'C(k, i).`

Read it as: to move k steps forward, take the differences at the point you are standing on and combine them with binomial coefficients. It holds for **every** function f and every k . There is no hypothesis on the degree. The reason is that shifting by one is $1 + \Delta$, so shifting by k is $(1 + \Delta)^k$, and the binomial theorem does the rest.

This is worth insisting on, because it is where the two algorithms of the paper come from. They are not two ideas. They are this one identity read at $k = 1$ and at general k . A degree hypothesis is needed only to stop the sum at d rather than at k , which is `dif_shift_deg`, and from it Newton's interpolation `newton`.

5 The difference table

The object the algorithms carry is the list of the first $d+1$ differences at the current argument.

Definition `dtab d f n : seq V := mkseq (fun i => difn i f n) d.+1.`

Its first entry is the value $f(n)$; the others are what is needed to move on. Take the paper's own example, $P(x) = x^3$, of degree 3, so four entries.

	$n = 0$	1	2	3	4	5
P	0	1	8	27	64	125
ΔP	1	7	19	37	61	
$\Delta^2 P$	6	12	18	24		
$\Delta^3 P$	6	6	6	6		

The table at $n = 0$ is $(0, 1, 6, 6)$. The bottom row is constant, which is `degle_const`: the d -th difference of a polynomial of degree d does not move. Below it everything is zero, and `nth_dtab` says that reading past the end of the table gives 0, which is the right answer rather than an accident.

6 The tabulated shift

One step of the paper's Figure 8 adds each row of the table to the row below it.

Definition tstep t : seq V :=
 mkseq (fun i => nth 0 t i + nth 0 t i.+1) (size t).

Lemma tstepE d f n : degE d f -> tstep (dtab d f n) = dtab d f n.+1.

On the table above: $(0, 1, 6, 6)$ becomes $(0 + 1, 1 + 6, 6 + 6, 6 + 0) = (1, 7, 12, 6)$, which is the column at $n = 1$. Only additions were used, and that is the whole point — on a GPU these are multi-precision additions and nothing else. Iterating gives the walk along consecutive intervals.

Lemma tstep_iter d f n k :
 degE d f -> iter k tstep (dtab d f n) = dtab d f (n + k)%N.

The proof of `tstepE` is the recurrence `dif_stepE`, which says $\Delta^i f(n+1) = \Delta^i f(n) + \Delta^{i+1} f(n)$, together with the remark about reading past the end: at the last row the missing term is 0 because the polynomial has degree d .

7 The straightforward shift

The same table can be moved k places in one go, by multiplying it by the upper triangular matrix whose (i, j) entry is $\binom{k}{j-i}$.

Definition sstep k t : seq V :=
 mkseq (fun i => \sum_(l < size t) (nth 0 t (i + l)%N) *+ 'C(k, l)) (size t).

Lemma sstepE d f n k :
 degE d f -> sstep k (dtab d f n) = dtab d f (n + k)%N.

On $(0, 1, 6, 6)$ with $k = 2$ the first entry is $0 \cdot 1 + 1 \cdot 2 + 6 \cdot 1 = 8$, the second $1 \cdot 1 + 6 \cdot 2 + 6 \cdot 1 = 19$, the third $6 \cdot 1 + 6 \cdot 2 = 18$, the fourth 6. That is the column at $n = 2$, reached in one step instead of two.

This costs multiplications, which is why the paper keeps it on the CPU. That one step of it is one tabulated step is `sstep1`.

8 The hybrid split

The paper's deployment does both: jump in packets on the CPU, then walk inside the thread.

Lemma hybridE d f Sz s nt :
 degE d f ->
 iter s tstep (sstep (nt * Sz)%N (dtab d f 0%N)) = dtab d f (nt * Sz + s)%N.

A shift by $tS + s$ is t straightforward steps of size S followed by s tabulated ones. Both sides are the table at the same argument, so the two halves of the computation agree by construction and not by a separate argument.

9 The hierarchical method

So far one table walks along the arguments. The paper's real move is to walk along the **intervals**. Write the argument as $x = kN + m$. For a fixed k , the function $m \mapsto P(kN + m)$ is a polynomial of degree at most d , so Newton's formula expands it in the binomial basis:

Definition acoef j k : V := difn j (fun m => P (k * N + m)%N) 0%N.

Lemma hierarchicalE k m :
 P (k * N + m)%N = \sum_(j < d.+1) acoef j k *+ 'C(m, j).

The $d + 1$ numbers $a_j(k)$ are the interval's polynomial. Now the claim that makes the method work: each a_j is itself a polynomial in k , of degree at most d . So it has its own difference table, and the

tables of the previous sections move it from interval k to interval $k + 1$. One shift by N has become $d + 1$ shifts by 1.

Take $P(x) = x^3$ and $N = 2$. Then

$$P(2k + m) = 8k^3 + (12k^2 + 6k + 1)\binom{m}{1} + (12k + 6)\binom{m}{2} + 6\binom{m}{3},$$

and the four coefficients have degrees 3, 2, 1, 0 in k . At $k = 1$, $m = 1$ this reads $8 + 19 = 27$, which is 3^3 .

The degrees falling by one along that row is the heart of the matter, and it is the step the paper states without proof. In Rocq it is a lemma of its own.

Definition `difh h f : nat -> V := fun n => f (n + h)%N - f n.`

Lemma `difh_deg e h f : degle e.+1 f -> degle e (difh h f).`

A difference over a stride of h arguments lowers the degree by one. The reason is Newton again: written out, a shift by h is a combination of the differences of order 1 to h , and the constant term is absent. Iterating it $d + 1$ times annihilates a polynomial of degree d (`difhn_deg`), and stepping k by one turns P into its shift by N (`difn_acoef`). Put together, they give `acoef_deg`: every a_j has degree at most d in k .

10 The polynomial of the search

`ShiftExp.v` runs the machinery on the polynomial the search really needs.

The paper’s own deployment uses a Taylor polynomial of degree 2 over intervals of 2^{15} arguments. That is far short of what a search for cases with 35 identical bits requires. We take instead the polynomial `Cheb.v` already certifies: the degree 7 near-minimax approximation of `exp` over $[0.25, 0.25001)$, within 2^{-160} of `exp` on the whole interval, and `htr.c`’s own interval length $N = 2^{20}$.

`Cheb.v` writes that polynomial over the reals, as $(\sum_{k \leq 7} A_k(x - c)^k) / 2^{220}$. The shifts want integers. On the grid the argument is $x_n = (\text{x0num} + n) / 2^{54}$, so $x_n - c$ is $(\text{dlt} + n) / 2^{54}$ with $\text{dlt} = \text{x0num} - \text{Cc}$, and clearing every denominator gives an integer:

Definition `Pdir (n : nat) : Z :=`
`\sum_(0 <= k < 8) Ascaled k * (dlt + n%:R) ^+ k.`

where `Ascaled k` is $A_k \cdot 2^{54(7-k)}$ and the scale is $2^{598} = 2^{220+7 \cdot 54}$. A value of `Pdir` is about 600 bits, nineteen words of 32 bits — the fixed-size multi-precision words of the paper.

From there everything is an instance of what came before: `Pdir_deg` gives the eight-entry table, `PdirE` the interval polynomials, `aexp_deg` their coefficients as polynomials in the interval index, `aexp_tab` and `aexp_hybrid` the two ways of generating them.

The last statement is the one worth the trouble.

Theorem `Pdir_exp (n : nat) :`
`Z.lt (Z.of_nat n) nargs ->`
`Rabs (exp (xgrid n) - IZR (Pdir n) / 2 ^ vden) <= / 2 ^ 160.`

Every value the shifts produce, at every argument of the search, is within 2^{-160} of the exponential. There is one approximation in the whole development and it was certified once, by `Cheb.cheb_valid`. Because the shifts do not round, that single certificate covers all 171 799 interval polynomials. No error is accumulated and none has to be analysed.

11 The cost

`Shift.v` is 513 lines, `ShiftBridge.v` 48 and `ShiftExp.v` 190. Nothing in them is admitted, and `Pdir_exp` rests on exactly the 55 axioms that `Cheb.cheb_valid` already carries. Those are 25 declarations of Rocq’s primitive 63-bit machine operations, the 26 specifications that say each agrees with $\mathbb{Z}$ arithmetic modulo 2^{63} , and the 4 classical axioms `Stdlib`’s real numbers are built on. They are the price of `Reals` and `CoqInterval`, and `Pdir_exp` adds none of its own.

The one identity that had to be done by hand is the last one, and it is the least interesting.

Lemma `Pdir_chebE` $(n : \text{nat}) : \text{IZR } (\text{Pdir } n) / 2 ^ \text{vden} = \text{P_R } (\text{xgrid } n).$

The integer the shifts carry, divided by its scale, is the polynomial of `Cheb.v` at that grid point. Both sides are the same polynomial; the work is to move the powers of two from inside the integer out into the denominator, eight times. For one term this is `term_bridge`, and the content of it is an exponent count: the integer holds $54(7 - k)$ powers of two and the denominator 598, what is left is $220 + 54k$, and $54(7 - k) + 220 + 54k = 598$ whatever k is.

That lemma lives in `ShiftBridge.v`, which loads no `mathcomp` on purpose. The statement is about $\mathbb{Z}$ and $\mathbb{R}$ and nothing else, and with `mathcomp` loaded the rewriting lemmas on powers pick up the wrong subterms. Keeping it apart also makes it compile in a second and a half rather than twenty five.

Nothing in the development computes a value of the polynomial. There is no `vm_compute`, no `Eval` and no `native_compute`. The only arithmetic anywhere is `field` normalising powers of two, and the whole of `ShiftExp.v` outside its `Require` lines measures 4.2 seconds: 2.1 for the eight applications of `term_bridge`, 1.0 for the final `field`.

The integers are Rocq’s binary `Z`, given its abelian group structure by `mathcomp`’s `ssrZ`, and not `mathcomp`’s `int`, which is built on unary natural numbers. For the same reason the number of arguments, 180 143 985 095, is kept in `Z`: as a `nat` it would be that many successors, and any tactic asked to decide a bound against it by computation would not come back.